Implementation and Verification of a CAN-FD Bus Transceiver in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

Abstract 2

Abbreviations 2

1 Introduction 3
- 1.1 Motivation 3
- 1.2 Existing CAN Controller 3
  - 1.2.1 Limitations 4
  - 1.2.2 Decision to Redesign 5
- 1.3 Existing CAN FD IP Cores 5
  - 1.3.1 Open-Source Implementations 5
  - 1.3.2 Commercial Implementations 5
  - 1.3.3 Rationale for In-House Development 7
- 1.4 Problem Statement 7
- 1.5 Objectives 8

2 Background 8
- 2.1 CAN Protocol Evolution 8
- 2.2 ISO 11898-1:2024 Standard 8
- 2.3 VHDL and OSVVM 8

3 Verification Planning 8
- 3.1 Overview and Scope 8
- 3.2 Requirement Taxonomy 8
- 3.3 Observability Classification 9
  - 3.3.1 Canonical Layer Interfaces 9
  - 3.3.2 Classification Rules 10
- 3.4 Verification Plan Construction 11
  - 3.4.1 AI-Assisted Extraction 11
  - 3.4.2 Human-in-the-Loop Validation 11
- 3.5 Storage Format and Tooling 11
- 3.6 Verification Strategy 12

4 Design and Architecture 13
- 4.1 Design Space Exploration 13
  - 4.1.1 Company Design Constraints 13
  - 4.1.2 Monolithic vs. Layered Architecture 14
  - 4.1.3 Combined vs. Separated TX/RX FSMs 14
  - 4.1.4 Per-Field vs. Per-Phase FSM Granularity 15

4.1.5 Interface Record Design 15
4.1.6 Dual CRC Data Feeds 16
4.2 System Overview 16
4.3 LLC Frame Format 16
4.4 Interface Definition Tables 18
4.5 Protocol-Driven Type System 27
4.6 LLC Sub-layer 27
4.6.1 `can_llc_tx` 27
4.6.2 `can_llc_rx` 27
4.7 MAC Sub-layer 27
4.7.1 `can_mac_tx` 28
4.7.1.1 `can_mac_tx` Internal Interfaces 28
4.7.1.2 `can_mac_fsm_tx` 31
4.7.1.3 `can_mac_ser_tx` 32
4.7.1.4 `can_mac_bs` 32
4.7.1.5 `can_mac_crc` 32
4.7.2 `can_mac_rx` 35
4.7.3 `can_mac` Unified Wrapper 35
4.8 FCE Sub-layer 38
4.9 PCS Sub-layer 38
4.9.1 `can_pcs_tx` 38
4.9.2 `can_pcs_rx` 38

5 Implementation **38**
5.1 Type Safety and Packages 38
5.2 Bit Timing and TDC 38
5.3 CRC and Bit Stuffing 41

6 Verification and Results **41**
6.1 Testbench Results Summary 41

7 Discussion **41**

8 Conclusion **41**

9 References **41**

Abstract

This document is a work-in-progress draft for the B.Eng thesis. Currently, the project is in the ***Verification Plan*** *phase (Phase 2), with architectural prototyping for the transmitter sub-system completed.*

This thesis describes the design, implementation, and verification of a CAN (Controller Area Network) and CAN-FD (Flexible Data rate) transmitter sub-system. The implementation is targeting high-reliability engine controller applications and is compliant with the ISO 11898-1:2024 standard [1]. Key features include support for both Classic and FD frame formats, Transmitter Delay Compensation (TDC) for high-speed data phases, and a modular architecture separated into Link Layer Control (LLC), Media Access Control (MAC), and Physical Coding Sublayer (PCS).

Abbreviations

Table 1: Abbreviations used in this report.

Abbreviation	Meaning
ACK	Acknowledgement
AF	Acceptance Field
AUI	Attachment Unit Interface
CAN	Controller Area Network
CBFF	Classical Base Frame Format
CEFF	Classical Extended Frame Format
DF	Data Frame
DLL	Data Link Layer
EF	Error Frame
FBFF	FD Base Frame Format
FCE	Fault Confinement Entity
FEFF	FD Extended Frame Format
FTYP	Frame Type
LLC	Logical Link Control
LSDU	LLC Service Data Unit
MAC	Medium Access Control
OF	Overload Frame
OSI	Open Systems Interconnection
PCS	Physical Coding Sublayer
PDU	Protocol Data Unit
PMA	Physical Medium Attachment
RF	Remote Frame
SAP	Service Access Point
SDU	Service Data Unit
SOF	Start of Frame
TEC/REC	Transmit Error Counter / Receive Error Counter
VCID	Virtual CAN Channel Identifier

1 Introduction

1.1 Motivation

The Controller Area Network (CAN) has been the workhorse of automotive and industrial communication for decades. However, the increasing bandwidth requirements of modern systems led to the development of CAN-FD (Flexible Data rate), which allows for larger payloads and higher bit rates. This project aims to provide a robust, hardware-independent VHDL implementation of a CAN-FD transmitter.

1.2 Existing CAN Controller

The starting point for this project is an existing CAN Classic controller developed internally at the company. The controller is implemented in VHDL and has been integrated into a production IO-extender FPGA design.

It supports CAN Classic frames with both 11-bit (base) and 29-bit (extended) identifiers at bit rates up to 500 kbit/s, and has been verified through hardware bring-up on physical CAN buses.

The controller follows a monolithic architecture where a single top-level wrapper (`can_bus_controller`) instantiates six sub-modules: a combined TX/RX frame FSM (`can_fsm`), a bit timing generator (`can_node_clock`), a dynamic bit stuffer (`can_stuff_bit_gen`), a CRC-15 engine (`gen_crc`), and two Avalon-ST converters for frame serialization and deserialization (`can_ast_to_serial`, `can_serial_to_ast`). The entire design totals roughly 2,000 lines of VHDL across seven source files.

The central component is `can_fsm`, an 810-line state machine with 18 states that handles both transmission and reception in a single process. The FSM manages frame arbitration, bit-level transmission and reception, stuff bit error checking, CRC validation, ACK handling, and error flag generation. Error counting (TEC/REC) and node state transitions (Error Active, Error Passive, Bus Off) are handled in separate processes within the same file but are tightly coupled to the main FSM through shared signals.

1.2.1 Limitations

While the existing controller is functional for CAN Classic, several architectural limitations prevent it from being extended to support CAN FD:

Single bit rate domain. The `can_node_clock` module generates a single pair of timing strobes - one sample pulse and one transmit pulse - derived from a fixed set of bit timing parameters. CAN FD requires switching between a nominal bit rate (used during arbitration) and a faster data bit rate (used during the data phase), with Transmitter Delay Compensation (TDC) to account for the transceiver round-trip delay at the higher rate. Adding dual bit rate support and TDC to the existing clock module would require a fundamental redesign of the timing architecture.

Dynamic bit stuffing only. The `can_stuff_bit_gen` module implements the CAN Classic rule of inserting an inverse bit after five consecutive identical bits. CAN FD introduces a second stuffing mode - fixed bit stuffing - where a stuff bit is inserted at fixed intervals during the CRC field, and a Stuff Bit Count (SBC) field with Gray-coded parity is appended. The existing stuffer has no mechanism for mode switching or SBC generation.

Single CRC polynomial. The controller uses a single CRC-15 instance. CAN FD requires three CRC polynomials: CRC-15 for Classic frames, CRC-17 for FD frames with payloads up to 16 bytes, and CRC-21 for larger FD payloads. Furthermore, the CRC data feed differs between Classic and FD - in FD frames, dynamic stuff bits in the arbitration region are included in the CRC computation, requiring a dual data feed to the CRC engine.

Combined TX/RX FSM. The monolithic FSM interleaves transmission and reception logic in every state, with the `is_transmitter` flag selecting the active code path. This coupling makes it difficult to add FD-specific states (such as BRS, ESI, and the SBC field) without increasing the already high cyclomatic complexity. A CAN FD frame has more control fields than a Classic frame, and handling both TX and RX paths for all four frame formats (Classic Base, Classic Extended, FD Base, FD Extended) in a single process would result in an unwieldy state machine.

Embedded error handling. Error detection, error flag transmission, and error counter management are distributed across the main FSM process and its auxiliary processes. The ISO 11898-1 standard defines the Fault Confinement Entity (FCE) as a logically separate component with well-defined interfaces to the MAC and PCS sub-layers. Extracting the error handling into a reusable, independently testable FCE module - as required by the standard's layered architecture - would require significant refactoring of the existing FSM.

1.2.2 Decision to Redesign

Given these limitations, extending the existing controller to CAN FD would require modifying nearly every sub-module and fundamentally restructuring the FSM. The resulting design would carry the constraints of the original monolithic architecture while trying to support a significantly more complex protocol. Instead, a clean-sheet redesign was chosen, structured around the ISO 11898-1 layered architecture (LLC, MAC, PCS, FCE) with separated TX and RX paths, reusable sub-components (bit stuffer, CRC engine), and typed record interfaces. This approach allows each sub-layer to be implemented and verified independently, supports both Classic and FD frame formats from the outset, and produces a modular design that can be maintained and extended without cross-cutting changes.

1.3 Existing CAN FD IP Cores

Before committing to an in-house redesign, the available CAN FD controller IP cores were evaluated. Table 2 summarizes the candidates, spanning both open-source and commercial offerings.

1.3.1 Open-Source Implementations

CTU CAN FD [2], [3] is the only mature open-source CAN FD controller available as synthesizable HDL. Developed at the Czech Technical University in Prague, it is written in VHDL, licensed under MIT, and has been conformance-tested against ISO 16845-1 [4]. The controller includes a full TX and RX pipeline with up to four TX buffers, acceptance filtering, timestamping, and a register interface with DMA support. A mainline Linux kernel driver has been available since kernel version 5.12. CTU CAN FD represents a complete, production-oriented CAN node - a significantly broader scope than what is needed in this project.

OpenCores CAN [5] is a Verilog controller modeled after the Philips SJA1000 register interface. It is one of the earliest open-source CAN cores and is widely cited in academic work. However, it supports only CAN 2.0B (Classic CAN) and has seen no active development since approximately 2010. It cannot serve as a starting point for CAN FD.

Canola [6] is a VHDL CAN 2.0B controller with a clean VHDL-2008 codebase and a cocotb-based testbench. It includes a Triple Modular Redundancy (TMR) wrapper for radiation-tolerant applications. Like the OpenCores core, it does not support CAN FD.

1.3.2 Commercial Implementations

Bosch M_CAN [7], [8] is the reference CAN FD controller, developed by the inventor of both CAN and CAN FD. M_CAN is the IP core embedded in virtually every automotive microcontroller (NXP S32, Infineon AURIX, STM32, TI Jacinto, Renesas RH850). It is licensed under NDA with per-design royalty fees.

AMD/Xilinx CAN FD [9] is a soft IP core included in the Vivado Design Suite. It provides an AXI4-Lite register interface with up to 32 acceptance filters, TX mailboxes, and RX FIFOs. It is device-locked to AMD/Xilinx FPGAs and cannot be ported to other targets.

CAST CAN FD [10] is a technology-independent RTL core with AMBA APB/AHB bus interface options. It is licensed per-design with an upfront fee. Synopsys (DesignWare) and Cadence offer similar ASIC-targeted CAN FD cores under their respective IP licensing programs.

Table 2: Survey of available CAN FD controller IP cores.

Implementation	Language	CAN FD	License	Scope	Conformance Tested
CTU CAN FD [2]	VHDL	Yes	MIT	Full node (TX+RX, buffers, DMA)	ISO 16845-1
OpenCores CAN [5]	Verilog	No	LGPL	Full node (CAN 2.0B only)	No
Canola [6]	VHDL	No	MIT	Full node (CAN 2.0B, TMR)	No
Bosch M_CAN [7]	HDL (NDA)	Yes	Per-design royalty	Full node	Yes (reference)
AMD/Xilinx CAN FD [9]	HDL	Yes	Vivado-included	Full node	Yes
CAST CAN FD [10]	RTL	Yes	Per-design fee	Full node	Yes

1.3.3 Rationale for In-House Development

Despite the availability of these solutions, none satisfies the combined requirements of a large engine design company developing safety-critical control systems. The decision to develop the CAN FD transceiver in-house was driven by five considerations.

IP ownership and supply chain independence. Commercial IP cores introduce a licensing dependency on an external vendor. For a product with a multi-decade lifecycle - typical in heavy-duty engine applications - this dependency carries risk: vendors may discontinue support, change licensing terms, or be acquired. Owning the RTL outright eliminates this exposure and ensures that the design can be maintained, ported, and modified without third-party approval for the full product lifetime. The open-source CTU CAN FD avoids the licensing risk, but using it still means adopting a codebase whose architecture, naming conventions, and design decisions were made for a different context.

Verification authority. In safety-critical domains, the verification evidence must be traceable from standard requirements to RTL assertions and testbench results. Adopting a third-party core - even one conformance-tested against ISO 16845-1 - means inheriting its verification artifacts rather than producing them. The company's verification methodology requires full control over the verification plan, the testbench architecture, and the assertion coverage. Building the RTL in-house allows the verification plan (described in Section 3) to drive the implementation, ensuring that every module is verified against the specific requirements extracted from the standard, using the company's own toolchain and conventions.

Architectural scope. All available IP cores implement a complete CAN node: TX and RX pipelines, message RAM, acceptance filtering, buffer management, register interfaces, and in some cases DMA controllers. The company's application requires only the data link layer (LLC, MAC, FCE) and physical coding sublayer (PCS) - the protocol engine that converts between byte-level frame data and the serial bus. The higher-level buffering and filtering logic already exists in the company's FPGA infrastructure. Adopting a full-node IP core would introduce unnecessary complexity and area overhead, and the integration effort to bypass or disable the unused subsystems may approach the effort of a targeted implementation.

Integration with existing infrastructure. The company's FPGA designs use a specific Avalon-ST streaming interface for inter-module communication, a particular clock and reset architecture, and established conventions for signal naming and module boundaries. A third-party core would require an adaptation layer to bridge its native interface (AXI, APB, or custom register map) to the existing infrastructure. The in-house design uses the company's interface conventions natively, eliminating this integration overhead.

Platform independence. The AMD/Xilinx CAN FD core is locked to Xilinx devices. The Bosch M_CAN and other commercial cores are delivered as technology-specific netlists or encrypted RTL for a particular target. The in-house design is written in portable VHDL-2008, synthesizable on any FPGA platform or ASIC flow, ensuring that the IP remains usable if the company changes FPGA vendors.

1.4 Problem Statement

The need for CAN FD support in the company's engine controller platform, combined with the architectural limitations of the existing CAN Classic controller (Section 1.2.1) and the unsuitability of available third-party IP cores for the company's specific requirements (Section 1.3.3), motivates the development of a new CAN FD transceiver from the ground up. The challenge lies in creating a modular, independently verifiable implementation that supports both Classic and FD frame formats, handles dual bit rate switching with Transmitter Delay Compensation (TDC), and maintains strict compliance with ISO 11898-1 [1] - all while fitting into the company's existing FPGA infrastructure and verification methodology.

1.5 Objectives

- Implement a VHDL-2008 compliant CAN/CAN-FD transmitter.
- Support both Base (11-bit) and Extended (29-bit) identifiers.
- Implement TDC measurement and compensation logic.
- Ensure high verification coverage using OSVVM and GHDL.

---

2 Background

2.1 CAN Protocol Evolution

Brief history from CAN 2.0 to CAN-FD.

2.2 ISO 11898-1:2024 Standard

Overview of the data link layer and physical signaling requirements [1].

2.3 VHDL and OSVVM

The role of modern VHDL standards and verification frameworks in digital design.

---

3 Verification Planning

3.1 Overview and Scope

Protocol compliance is the central objective of this project. The CAN and CAN-FD standards [1] specify hundreds of normative requirements governing frame structure, bit timing, error handling, and fault confinement. To verify the transmitter against these requirements systematically, a structured **Verification Plan** was developed as the first major deliverable of the project - before any RTL implementation began.

The plan covers the four in-scope frame formats: Classic Basic (CB), Classic Extended (CE), FD Basic (FB), and FD Extended (FE). CAN XL frames are excluded. In total, the plan contains 168 requirements extracted from the standard, organized by architectural layer (LLC, MAC, PCS, FCE) and spanning the major functional areas:

- **Frame structure**: Field ordering, bit-level encoding, and format-specific control bits.
- **CRC generation**: CRC-15 (Classic), CRC-17, and CRC-21 (FD) polynomials.
- **Bit stuffing**: Dynamic stuffing during arbitration/data and fixed stuffing in the FD CRC region.
- **Error handling**: Bit error, ACK error, Form error, and Stuff error detection.
- **Fault confinement**: TEC/REC counter management and Error Active/Passive/Bus Off state transitions.

3.2 Requirement Taxonomy

A key design decision was the development of a two-axis taxonomy to classify each requirement by its **shape** and **scope**. This taxonomy determines both *how* a requirement is verified and *what environment* is needed.

Shape classifies the verification primitive. These categories draw on established concepts from formal verification theory:

- **Triggered**: A precondition/event/postcondition triplet that translates directly into a directed test procedure - establish the precondition, apply the event, assert the postcondition. This structure follows the Hoare triple formalism {P} S {Q} [11].
- **Invariant**: A property that must hold at all times (e.g., "the SOF bit shall always be dominant"), mapping to concurrent assertions or monitors.
- **Liveness**: A property asserting that something eventually happens (e.g., "after detecting an error, the node shall eventually transmit an error flag"), requiring temporal reasoning [12].
- **Reachability**: A property asserting that a state or condition *can* be reached (e.g., "the node shall be capable of entering Bus Off"), mapping to coverage points.

Scope defines the required verification environment:

- **Frame**: Verified by inspecting a single transmitted bit-stream in isolation.
- **Node**: Requires visibility into internal state such as error counters or FSM transitions.
- **Bus**: Requires a multi-node simulation with arbitration and acknowledge behavior.

Together, these two axes allow each requirement to be mapped to the appropriate testbench level and verification technique without ambiguity.

3.3 Observability Classification

The shape and scope taxonomy determines *how* and *where* a requirement is tested, but it does not answer a more fundamental question: *can* the requirement be tested at all from outside the design under test? Not every normative requirement produces a visible effect at the module's ports. Some requirements constrain internal counter thresholds, define structural concepts, or restrict valid configurations - none of which produce a unique, externally distinguishable output. To distinguish these cases systematically, a third classification axis - **observability** - was introduced.

Observability is assessed **per layer**, not per top-level CAN node. The design under test for a PCS requirement is the PCS module in isolation; for a MAC requirement it is the MAC module. This layer-relative framing is essential because a signal that is internal within a full node may be perfectly visible at a sub-layer boundary. For example, the sample point strobe is internal to the CAN node as a whole, but it is the defining output of the PCS module - it determines *when* `PCS_Data.Indicate` fires at the MAC↔PCS boundary and is therefore directly observable in a PCS-level testbench.

3.3.1 Canonical Layer Interfaces

To make observability judgments repeatable and independent of VHDL implementation choices, the classification is anchored in the **canonical service primitives** defined by ISO 11898-1 [1]. The standard specifies inter-layer boundaries as abstract service access points with named primitives and parameters:

- **LLC ↔ User**: `L_Data.Request`, `L_Data.Confirm`, `L_Data.Indication` - carrying frame content, transfer status, and timestamps (§6.4.5).
- **MAC ↔ PCS**: `PCS_Data.Request(Output_Unit)`, `PCS_Data.Indicate(Input_Unit)` - the bit-level transmission and reception interface; `PCS_Status.Transmitter(D_Transmit)` and `PCS_Status.Receiver(D_Receive)` - signalling the FD data phase (§7.2).

- **MAC ↔ FCE**: Error notifications (`Error(type)`, `Successful_transfer`), state transition requests (`Error_passive_request`, `Error_active_request`), and responses (`Error_passive_response`, `Error_active_response`) (§8.1.3, Tables 16-17).
- **PCS ↔ FCE**: `Bus_off_request`, `Bus_off_release_request` and their responses (§8.1.3, Tables 18-19).

These canonical interfaces serve as the reference frame for observability: a requirement is classified based on whether its postcondition manifests at the relevant layer's canonical boundary, regardless of how the VHDL implementation names its ports or structures its handshaking.

In addition to the interface primitives, each layer has a set of **configurable values** that the testbench knows at instantiation time - entity generics, constants, or driven stimulus. For the PCS, these include the nominal and FD data bit timing parameters from ISO Table 12 (prescaler, propagation segment, phase segments, SJW) and TDC settings (enable flag, SSP offset). For the MAC, the relevant configurations are error signalling enable and protocol exception enable. For the FCE, there are no user-configurable parameters; its behaviour is entirely determined by fixed counting rules and thresholds defined in §8.1.4.

The combination of canonical interfaces and known configurables provides a complete decision framework: if a postcondition is fully determined by boundary primitives, configuration generics, and driven stimulus, it is externally verifiable; if it additionally requires knowledge of an internal algorithm, it is derived; if it has no boundary manifestation at all, it is internal.

3.3.2 Classification Rules

From this framework, three classification rules were defined:

Rule 1 - External. A requirement is `external` if its postcondition is fully observable at the layer's own canonical boundary, in one of two forms. *Rule 1a*: the postcondition maps directly onto a named parameter of a canonical service primitive (e.g., "MAC shall present `dominant` Output_Unit when transmitting SOF" - `Output_Unit` is a parameter of `PCS_Data.Request`). *Rule 1b*: the postcondition manifests as the *timing* of a primitive call, and that timing is completely determined by configuration generics and stimulus inputs known to the testbench (e.g., the sample point position equals `brp × (sync_seg + prop_seg + phase_seg1)`, all configuration generics, so the testbench can predict and verify exactly when `PCS_Data.Indicate` fires).

Rule 2 - Derived. A requirement is `derived` if its effect manifests at the layer boundary, but verifying correctness requires knowledge of a **non-trivial internal algorithm** beyond reading configuration generics and measuring timing. The distinction between trivial and non-trivial is important: counting stimulus bits or applying fixed positional offsets is trivial (Rule 1b), while polynomial computation (CRC), state-dependent counter arithmetic (FCE error counters), or multi-step protocol state tracking is non-trivial (Rule 2). For example, CRC bits appear in `Output_Unit` calls at the MAC↔PCS boundary, but verifying their correctness requires applying the CRC-15, CRC-17, or CRC-21 polynomial to the preceding data - a non-trivial computation that goes beyond what is directly visible at the interface.

Rule 3 - Internal. A requirement is `internal` if its postcondition is a structural definition with no behavioural output (e.g., "the bit time consists of four segments"), a constraint on valid configuration inputs rather than on observable output behaviour (e.g., "Phase_Seg2 shall be ≥ IPT + SJW"), or has no manifestation at any layer boundary even indirectly (e.g., oscillator tolerance specifications that are physical constraints not testable in digital simulation).

Applying these rules to the 168 requirements yielded the following distribution: 106 external (63%), 40 derived (24%), and 22 internal (13%). The high proportion of externally observable requirements reflects the standard's emphasis on bit-level protocol behaviour, which by definition crosses layer boundaries. The internal requirements are concentrated in the PCS layer (configuration constraints and oscillator tolerances) and the FCE (structural definitions of counting concepts). Each requirement's rationale field records which rule was applied and which canonical interface primitive or configurable value justifies the classification, providing a fully auditable trail from ISO clause to testability assessment.

3.4 Verification Plan Construction

With the taxonomy and observability framework defined, the next step was to extract and classify the normative requirements from the ISO standard text. This was a two-stage process combining AI-assisted extraction with manual engineering review.

3.4.1 AI-Assisted Extraction

The initial extraction of normative "shall" and "should" statements was performed using a **Large Language Model (LLM)**. The ISO standard was provided as a searchable markdown document, and the LLM was prompted to identify normative clauses, extract their wording verbatim, assign a shape and scope classification, and format the result as TOML entries. This task is well-suited for LLMs because it involves processing large volumes of technical text, identifying structural patterns (normative vs. informative language), and reformatting unstructured prose into a consistent structured format. The use of AI significantly accelerated the initial drafting phase and reduced the risk of human oversight during the translation from standard text to a machine-readable plan.

3.4.2 Human-in-the-Loop Validation

The AI-generated extraction served only as a first draft. Every requirement - its wording, assigned shape, scope, layer, and flags - underwent a comprehensive **manual review**. This step was critical to:

- Verify the technical accuracy of the AI's interpretation of protocol nuances (e.g., distinguishing between transmitter-side and receiver-side obligations).
- Refine the shape/scope classification where the standard's wording was ambiguous or where a single clause contained multiple independent requirements (flagged as `COMPOUND`).
- Identify requirements that depend on components outside the TX pipeline (flagged as `EXTERNAL_DEP`) or that are advisory rather than mandatory (flagged as `SHOULD`).
- Ensure that the resulting plan provides a reliable and authoritative basis for the subsequent VHDL implementation and testbench development.

3.5 Storage Format and Tooling

The Verification Plan is stored as a single TOML file (`verification_plan/verification_plan.toml`). TOML was selected for two practical reasons: (1) it is easy to read and edit by hand - each requirement is a self-contained `[[requirement]]` block with one key per line, making version-control diffs clean and merge conflicts rare; and (2) it can be parsed and manipulated programmatically by Python tools (specifically `tomlkit`, which preserves comments and formatting on round-trip). This second property was essential for building the automated tooling described below.

Each requirement entry carries metadata fields designed to drive the verification workflow:

Table 3: Verification-plan metadata fields and their intended use in workflow automation.

Field	Purpose
`shape`	Classification axis described in Section 3.2 - determines the verification primitive.
`scope`	Classification axis described in Section 3.2 - determines the test environment.
`layer`	Architectural sub-layer (LLC, MAC, PCS, or FCE), enabling a divide-and-conquer approach where each testbench targets one layer.
`precondition` / `event` / `postcondition`	For *triggered*-shape requirements, these three fields form a testable triplet that translates directly into a test procedure.
`coverage_target`	Describes how to verify the requirement (e.g., "assert CRC field matches polynomial output", "cover all four frame formats").
`observability`	Layer-relative testability classification (`external`, `derived`, `internal`) as defined in Section 3.3.
`observability_rationale`	Auditable justification citing the specific classification rule and canonical interface primitive.
`flags`	Marks special properties: `COMPOUND`, `AMBIGUOUS`, `EXTERNAL_DEP`, `SHOULD`, or `DOC_ONLY`. These flags inform review priority and test generation strategy.
`label` / `file`	Link the requirement to its implementing assertion label or testbench procedure, providing full traceability from standard clause to RTL.

To maintain the integrity of the plan as it evolves, a **Model Context Protocol (MCP) server** (`mcp_tools/verification_plan_manager.py`) was developed. The server exposes query, update, insert, delete, and statistics operations as tool calls that the AI coding agent can invoke directly within the development environment. Each write operation creates an automatic backup and validates field values against the schema before committing changes. This design serves two purposes: first, the agent receives summarized query results rather than the entire raw file, avoiding context window bloat; and second, constraining the agent to narrow, validated operations minimizes the risk of data corruption and hallucination - rather than asking the LLM to rewrite a large structured file (where it may silently drop entries, fabricate field values, or break TOML syntax), each MCP call targets a single atomic change with schema-level validation, making such errors structurally impossible.

3.6 Verification Strategy

The verification plan drives a layered testing strategy, where each level targets a different scope of requirements. The observability classification directly informs this strategy: *external* requirements can be verified through port-level stimulus and observation alone, *derived* requirements additionally need a reference model (e.g., a software CRC or error counter model) to compute the expected result, and *internal* requirements are verified through design review or configuration validation rather than simulation.

- **Unit Testing**: Individual modules (Serializer, CRC, Bit Stuffer) are verified in isolation against *frame*-scope requirements. External requirements at this level are verified by driving stimulus and checking output bits; derived requirements (e.g., CRC correctness) use a software reference model for comparison.

- **Protocol Testing**: `tx_can_protocol_tb` verifies frame structure and field timing, covering *node*-scope requirements that involve FSM state and internal counters. Derived FCE requirements (error counter thresholds) are tested by injecting controlled error sequences and observing the resulting state transitions at the MAC↔FCE boundary.
- **Integrated Testing**: `tx_can_tb` verifies end-to-end transmission, retries, and abort scenarios, targeting *bus*-scope requirements that involve multi-node arbitration and acknowledgment.

---

4 Design and Architecture

4.1 Design Space Exploration

Before settling on the final architecture, several design alternatives were evaluated. The exploration drew on three sources: the existing in-house CAN Classic controller (Section 1.2), the open-source CTU CAN FD core [2], [3], and the ISO 11898-1 standard's own layered reference model [1]. This section documents the key design decisions, the alternatives that were considered, and the constraints that shaped the outcome.

4.1.1 Company Design Constraints

The company imposes a set of non-negotiable constraints on all FPGA IP modules. These constraints were established before the design exploration began and are not subject to trade-off analysis - they define the feasible design space.

`std_logic`-only entity ports. All module interfaces must use `std_logic` and `std_logic_vector` exclusively. No custom enumeration types, booleans, or integers are permitted on entity ports. This mandate ensures that every module can be integrated into the company's existing synthesis and static analysis toolchain without adapter logic. The constraint precludes designs that expose protocol-level enumerations (such as FSM state types or frame format selectors) on their interfaces - a pattern used in the existing in-house controller, where `t_fsm_can_state` and `t_node_state` enums appear on debug ports. Internally, modules may use any VHDL-2008 type; the restriction applies only at module boundaries.

Per-module directory structure. Each module resides in its own directory with `hdl_src/`, `hdl_tb/`, and `test_case/` subdirectories. This convention is enforced across all company FPGA projects and must be followed by any new IP. It rules out monolithic file organizations where multiple entities share a single source directory.

Avalon-ST streaming interfaces. Data-transfer interfaces between modules use the Avalon-ST protocol (data, valid, ready, sop, eop). The existing FPGA infrastructure relies on this protocol for inter-module communication, and the CAN transceiver must integrate natively. This constraint excludes IP cores with AXI, APB, or custom register-map interfaces (such as CTU CAN FD's Avalon-MM or the Xilinx core's AXI4-Lite) without an adaptation layer.

Shared `gen_crc` IP block. The company maintains a reusable, parameterized CRC generator (`gen_crc`) in its IP library. All CRC computation must instantiate this block rather than implementing polynomial division inline. The CRC wrapper module must therefore be a structural shell that instantiates `gen_crc` with the appropriate polynomial, initial value, and data width for each CRC variant.

4.1.2 Monolithic vs. Layered Architecture

The most fundamental architectural decision was whether to extend the existing monolithic controller or adopt a layered decomposition.

The existing controller uses a flat architecture: a single FSM (`can_fsm`, 810 lines) directly drives the bus output, reads the bus input, manages bit counting, handles stuff bit insertion, coordinates CRC computation, and updates error counters - all in one clocked process. This approach minimizes inter-module latency and signal fanout, since every decision is made in a single combinational cone. For CAN Classic, where the frame has at most two format variants (base and extended) and a single bit rate, the complexity is manageable.

CAN FD, however, introduces four frame formats, dual bit rates, fixed bit stuffing with SBC encoding, three CRC polynomials, and a richer error model. Extending the monolithic FSM to cover these features would require nested conditionals on the format type in nearly every state, increasing the cyclomatic complexity well beyond what is practical to verify or maintain. The existing controller's `s_arbitration` state already contains 70 lines of interleaved TX/RX logic for two frame formats; scaling this to four formats with additional control fields (BRS, ESI, SBC) would roughly triple the state's complexity.

The alternative - and the approach adopted - is to follow the ISO 11898-1 reference model, which decomposes the data link layer into LLC, MAC, and PCS sub-layers with a separate FCE. Each sub-layer has a well-defined service interface (described in Section 3.3.1), and each can be implemented and verified independently. This decomposition introduces inter-module interfaces and pipeline latency, but it confines format-specific complexity to the module where it belongs: the MAC FSM handles frame field sequencing, the bit stuffer handles stuff bit insertion and SBC generation, and the CRC engine handles polynomial computation. No module needs to know about all three concerns simultaneously.

CTU CAN FD [2] takes a middle path: it separates protocol processing (in a `CAN Core` block) from buffer management and register access, but the protocol core itself remains a large monolithic unit. This is a reasonable trade-off for a general-purpose IP core that must support message filtering, multiple TX buffers, and DMA - features that require tight coupling between the protocol engine and the buffer subsystem. For the narrower scope of this project (protocol engine only, no message RAM or filtering), the full ISO layered decomposition is feasible and preferred because it directly maps each module to a testable subset of the standard's requirements.

4.1.3 Combined vs. Separated TX/RX FSMs

The existing controller uses a single FSM for both transmission and reception, switching between roles via an `is_transmitter` flag. Every state contains parallel `if transmit_i` and `elsif sample_rx_i` branches, one for the transmitter path and one for the receiver path. This approach has two advantages: it avoids duplicating shared state (bit counters, format detection) and it naturally handles the transmitter-to-receiver role switch that occurs when a node loses arbitration.

The disadvantage is that TX and RX logic evolve together. Adding a TX-only feature (such as the data-phase bit rate switch) requires modifying states that also contain RX logic, and vice versa. In CAN FD, the TX and RX paths diverge more than in CAN Classic: the transmitter drives BRS and ESI based on local state, while the receiver samples and validates them. The transmitter feeds the CRC engine with transmitted bits, while the receiver feeds it with received bits (including dynamic stuff bits in the arbitration region, per ISO 6.6.13.3). Encoding both paths in every state produces deeply nested conditionals that are difficult to review and difficult to cover in verification.

The chosen design uses separate TX and RX FSMs (`can_mac_fsm_tx` with 19 states, `can_mac_fsm_rx` with 17 states), each with its own bit stuffer and CRC interface. The two FSMs share no state. Arbitration loss in the TX FSM triggers a `transfer_status` change, but the actual reception of the frame is handled by the RX FSM operating independently on the same bus. This separation means that the TX FSM can be verified exhaustively against TX-side requirements without any RX stimulus beyond bus-level loopback, and conversely for the RX FSM.

The cost of separation is that sub-components used by both paths - the bit stuffer (`can_mac_bs`) and CRC engine (`can_mac_crc`) - must be either shared with multiplexing logic or duplicated. The chosen design duplicates both: `can_mac_tx` instantiates its own `can_mac_bs` and `can_mac_crc`, and `can_mac_rx` does the same. This trades a modest increase in area for complete independence between the two paths - each FSM drives its bit stuffer and CRC engine through its own interface signals without any arbitration or routing logic. The duplication is justified because the bit stuffer and CRC engine are small combinational-plus-register modules (under 120 and 80 lines respectively), while a shared-resource approach would require multiplexing logic in the `can_mac` wrapper and careful coordination to prevent one path's state from corrupting the other's. The `can_mac` wrapper (Section 4.7.3) therefore contains no datapath logic at all - it is purely structural, instantiating `can_mac_tx`, `can_mac_rx`, and `can_fce`, and merging only the FCE error signals.

4.1.4 Per-Field vs. Per-Phase FSM Granularity

A second FSM design decision was the granularity of states. The existing controller uses per-phase states: `s_arbitration` covers all ID bits, SRR, IDE, and RTR; `s_control` covers reserved bits and DLC; `s_data` covers all data bytes. Within each state, a bit counter and conditional logic dispatch the correct action for each bit position.

This per-phase approach keeps the state count low (18 states in the existing controller) but pushes complexity into the bit-counting logic. The `s_arbitration` state must distinguish between base and extended formats using the bit counter, handle the SRR/IDE branching point at bit 12/13, and detect arbitration loss - all in a single state with a single counter.

The alternative is per-field states, where each protocol field (SOF, ID, SRR/RRS, IDE, FDF, RES, BRS, ESI, DLC, Data, SBC, CRC, ACK, EOF) has its own state. This increases the state count (19 TX states, 17 RX states) but eliminates most bit-counter conditionals: when the FSM is in `s_brs`, it knows exactly which bit it is processing without consulting a counter. Format-dependent transitions are handled by the state graph itself - for example, the FSM transitions from `s_ide` to `s_fdf_r1_r0` for all formats, but from `s_fdf_r1_r0` it may go to `s_dlc` (Classic) or `s_res_r0` then `s_brs` (FD). Each state contains only the logic relevant to its specific field, and named guard predicates (`v_in_arbitration_field`, `v_in_dynamic_stuff_field`, `v_in_fixed_stuff_field`) replace repeated bit-range comparisons.

The per-field approach was chosen because it aligns with the verification plan: each field in the frame structure corresponds to a set of requirements in the verification plan, and each FSM state can be traced directly to those requirements. It also simplifies formal verification, since PSL assertions can reference state names rather than counter ranges.

4.1.5 Interface Record Design

The company's `std_logic`-only port constraint does not preclude the use of VHDL record types on entity ports - records of `std_logic` and `std_logic_vector` fields satisfy the constraint. The design uses typed record interfaces (e.g., `t_can_mac_pcs_if_m2s`, `t_can_mac_fsm_bs_if_s2m`) to bundle related signals into a single port. Each record type has a corresponding reset constant (e.g., `c_can_mac_pcs_if_m2s_reset`), ensuring that every module can be reset to a known state without manually enumerating each field.

The alternative - flat port lists with individual `std_logic` signals - was used in the existing controller, where the FSM entity has 20 individual ports for its various sub-module interfaces. This approach becomes unwieldy as the number of inter-module signals grows: the new design has over 60 inter-module signals across its interfaces, and bundling them into records reduces the port list from dozens of lines to six record-typed ports per FSM entity.

The naming convention follows the data-flow direction: `m2s`/`s2m` (master-to-slave/slave-to-master) for control interfaces, and `s2d`/`d2s` (source-to-destination/destination-to-source) for Avalon-ST data-transfer interfaces. This convention is consistent with the company's existing interface naming and makes the direction of data flow explicit at every port.

4.1.6 Dual CRC Data Feeds

A non-obvious design decision concerns the CRC engine's data input. In CAN Classic, the CRC is computed over the raw bit stream excluding stuff bits. In CAN FD, the CRC is computed over the raw bit stream in most of the frame, but in the arbitration region the computation includes dynamic stuff bits - a difference from Classic that exists because the FD CRC must protect the stuff bit count as well as the data. This means that a single data feed to the CRC engine is insufficient: the Classic CRC needs the de-stuffed stream, while the FD CRC needs the stuffed stream during arbitration and the de-stuffed stream elsewhere.

The chosen solution exposes two data inputs on the CRC interface: `data_cc` (Classic CAN data, always de-stuffed) and `data_fd` (FD data, which includes dynamic stuff bits during the arbitration region). The FSM drives both feeds, and the CRC wrapper routes `data_cc` to the CRC-15 engine and `data_fd` to the CRC-17 and CRC-21 engines. This avoids multiplexing logic inside the CRC module and keeps the CRC wrapper purely structural - it instantiates three `gen_crc` blocks and an output mux, with no protocol knowledge.

4.2 System Overview

A complete CAN node decomposes into a TX path and an RX path, coordinated by a shared Fault Confinement Entity (FCE) and Physical Medium Attachment (PMA) control, as shown in Figure 1. Each path spans three sub-layers - LLC (Section 4.6), MAC (Section 4.7), and PCS (Section 4.9) - with the LLC frame format defined in Section 4.3 and interface bundles defined in Section 4.4. A centralized types package (Section 4.5) defines all protocol constants and interface records. Within the MAC sub-layer, a unified `can_mac` wrapper (Section 4.7.3) instantiates `can_mac_tx`, `can_mac_rx`, and `can_fce`, merging their error signals internally so that the wrapper exposes only LLC and PCS interfaces for each path plus the FCE's LLC and PCS interfaces.

4.3 LLC Frame Format

The `LLC Frame` format used by the existing CAN-bus implementation (`can_bus_controller`) is depicted in Figure 2 with the ID byte format depicted in Table 4. A revised `LLC Frame` supporting FD is depicted in Figure 3. The revised format adds a 3-bit `FMT` [1, Sec. 6.4.3] field to the DLC byte (`LLC Frame` byte 4), expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags.

The `FMT` encodes the supported frame formats as '`000`' = CB, '`100`' = CE, '`010`' = FB, '`110`' = FE. Accordingly,

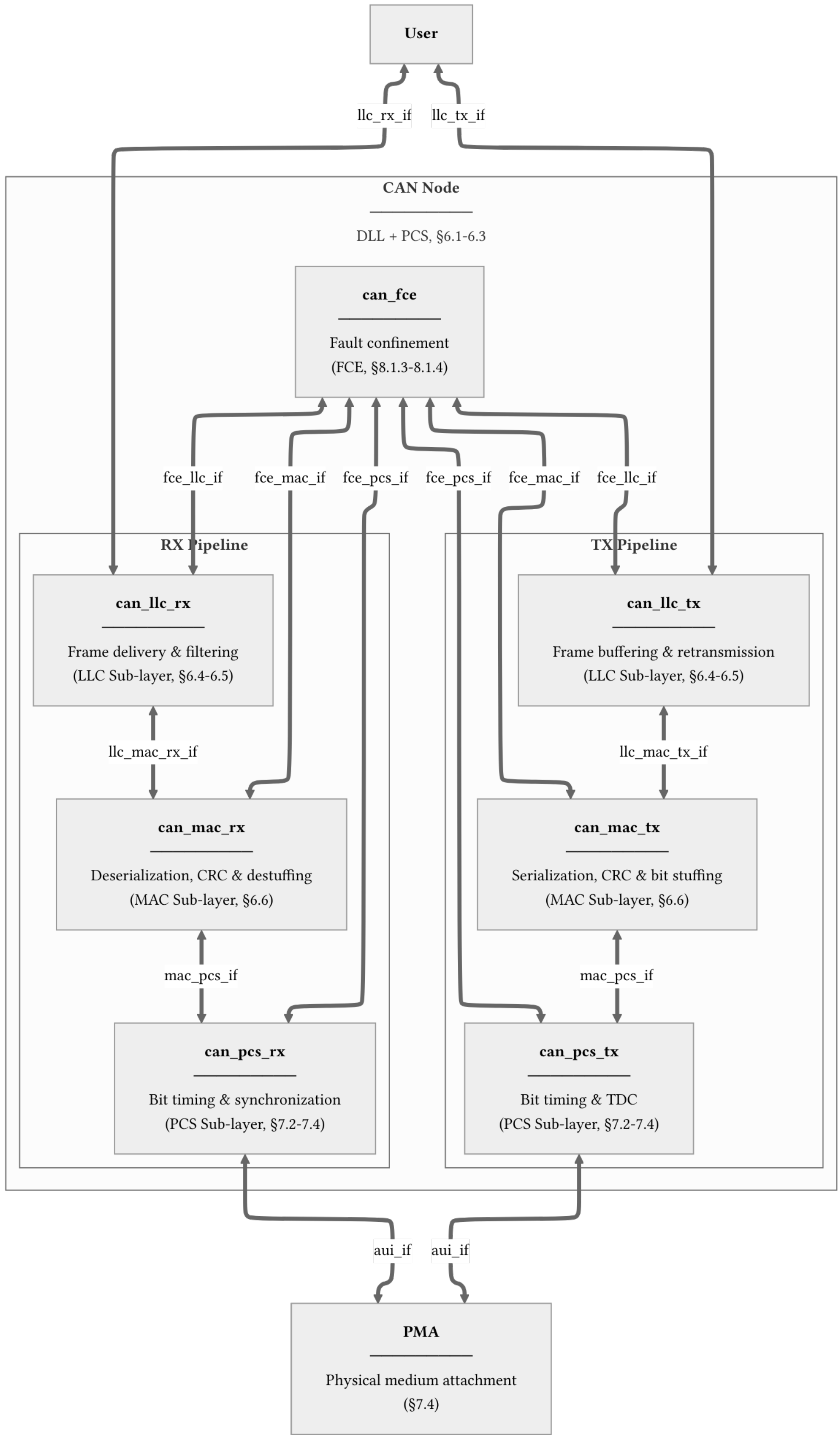

Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for llc_tx_if (Table 5), llc_rx_if (Table 6), llc_mac_tx_if (Table 7), llc_mac_rx_if (Table 8), mac_pcs_if (Table 9), aui_if (Table 10), fce_llc_if (Table 11), fce_mac_if (Table 12), and fce_pcs_if (Table 13).

for the frame content to be self-consistent the IDE bit must be set to `0` for `FMT` = CB/FB and `1` for `FMT` = CE/FE. The revised format is designed to be backward compatible. An implementation that only supports Classic frames can simply ignore the `FMT` bits and treat all frames as Classic, while an implementation that supports FD can use the `FMT` field and the additional control bits (BRS and ESI) to distinguish frame types without affecting the existing ID and data field structure.

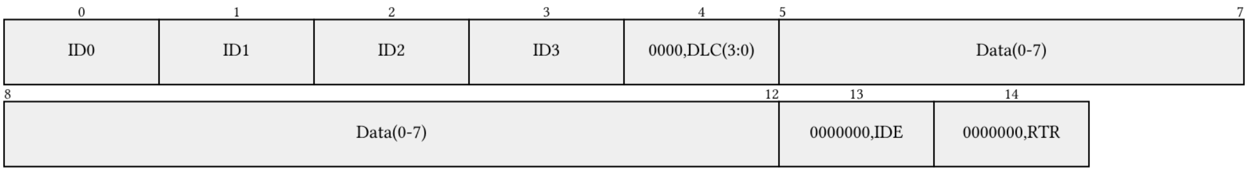

Figure 2: Current LLC frame format (15 bytes). ID byte encoding is defined in Table 4.

Figure 3: Revised LLC frame format with FD support, shown at maximum length (71 bytes). The FMT field selects frame type; BRS and ESI repurpose reserved bits in the final byte. ID byte encoding is defined in Table 4.

Table 4: ID byte encoding for the LLC frame formats shown in Figure 2 and Figure 3.

Format	Byte ID0	Byte ID1	Byte ID2	Byte ID3
Basic	`'00000000'`	`'00000000'`	`'00000' & 'ID(10-8)'`	`'ID(7-0)'`
Extended	`'000' & 'ID(28:24)'`	`'ID(23-16)'`	`'ID(15-8)'`	`'ID(7-0)'`

4.4 Interface Definition Tables

The interface bundles shown in Figure 1 are defined in full below, with each signal mapped to its corresponding ISO 11898-1 service primitive, [1]. This normative anchoring provides a direct traceability path from protocol clauses to VHDL ports. The types used in these interfaces are defined in Section 4.5.

All module interfaces use `std_logic` and `std_logic_vector` exclusively, as mandated for synthesis compatibility. Protocol semantics such as polarity, frame format, and transfer status are encoded as named `std_logic`/`std_logic_vector` constants defined in `pk_can_types` (Section 4.5). The LLC-User interface (`llc_tx_if`, `llc_rx_if`) is implemented as an Avalon-ST byte stream to maintain backward compatibility with the existing CAN bus controller, which uses the same `valid`/`ready`/`startofpacket` handshake convention.

Table 5: Interface definition for `llc_tx_if`. Implements `L_Data.Request` as an Avalon-ST byte stream. `L_Data.AbortRequest` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`), `llc_tx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` asserted on first byte
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`)	`valid` high; `sop` and `eop` deasserted on intermediate bytes
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`), `llc_tx_if.eop` (`std_logic`)	`valid` high with byte on data; `eop` asserted on last byte
6.4.5.5.2	`L_Data.Request`		Flow control	`LLC -> User`	`llc_tx_if.ready` (`std_logic`)	Asserted by LLC when able to consume next byte
6.4.5.5.3	`L_Data.AbortRequest`	`Handle`	Abort pending frame transfer	`User -> LLC`	`llc_tx_if.abort_request` (`std_logic`)	Pulse when user wants to cancel an in-progress transfer

Table 6: Interface definition for `llc_rx_if`. Implements `L_Data.Indication` as an Avalon-ST byte stream. `Timestamp` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`), `llc_rx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` asserted on first byte

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`)	`valid` high; `sop` and `eop` deasserted on intermediate bytes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`), `llc_rx_if.eop` (`std_logic`)	`valid` high with byte on data; `eop` asserted on last byte
6.4.5.5.5	`L_Data.Indication`		Flow control	`User -> LLC`	`llc_rx_if.ready` (`std_logic`)	Asserted by user when able to consume next byte
6.4.5.5.4	`L_Data.Confirm`	`Transfer_Status`, `Timestamp`, `Handle`	Report outcome of prior `L_Data.Request`	`LLC -> User`	`llc_rx_if.transfer_status` (`std_logic_vector(2:0)`)	Issued on frame completion, loss, or error

Table 7: Interface definition for `llc_mac_tx_if`. Frame length is self-describing from the DLC config bytes; the MAC serializer does not consume `eop`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.3, 6.6.4.2	`DLL SDU`	`LLC Frame`	Transfer LLC frame to MAC for serialization	`LLC -> MAC`	`llc_mac_tx_if.data` (`byte_t`), `llc_mac_tx_if.valid` (`std_logic`), `llc_mac_tx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` marks first byte of new frame
6.3, 6.6.4.2	`DLL SDU`		Flow control	`MAC -> LLC`	`llc_mac_tx_if.ready` (`std_logic`)	Asserted by MAC serializer when able to consume next byte
6.6.4.2	Interface control information	`Transfer_Status`	Report frame transmission outcome	`MAC -> LLC`	`llc_mac_tx_if.transfer_status` (`std_logic_vector(2:0)`)	Updated by MAC on completion, loss, or error

Table 8: Interface definition for `llc_mac_rx_if`. Carries the reconstructed LLC frame from `can_mac_rx` to `can_llc_rx` (Section 4.7.2) as an Avalon-ST byte stream. The RX FSM stores received bits in an internal frame array during reception and streams the completed frame after EOF.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.6.4.3, 6.6.9	`DLL SDU`	Reconstructed LLC `Frame`	Transfer reconstructed LLC frame to LLC	`MAC -> LLC`	`llc_mac_rx_if.data` (`byte_t`), `llc_mac_rx_if.valid` (`std_logic`), `llc_mac_rx_if.sop` (`std_logic`), `llc_mac_rx_if.eop` (`std_logic`)	Avalon-ST byte stream; the RX FSM streams the stored `llc_frame` array during the quiet phase
6.6.4.3, 6.6.9	`DLL SDU`		Flow control	`LLC -> MAC`	`llc_mac_rx_if.ready` (`std_logic`)	Asserted by LLC when able to consume next byte

Table 9: Interface definition for `mac_pcs_if`. The `D_Transmit` status is signalled via a dedicated `use_data_rate` flag rather than encoding it in a semantic bit name. The PCS switches between nominal and data-phase bit timing based on this flag, keeping the interface to plain `std_logic` signals.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.2	`PCS_Data.Request`	`Output_Unit`	Present next bit for transmission	`MAC -> PCS`	`mac_pcs_if.polarity` (`std_logic`), `mac_pcs_if.valid` (`std_logic`)	MAC holds polarity stable; PCS samples at each bit boundary autonomously
7.2.1, 7.2.5	`PCS_Status.Transmitter`	`D_Transmit`	Indicate FD data-phase interval to PCS	`MAC -> PCS`	`mac_pcs_if.use_data_rate` (`std_logic`), `mac_pcs_if.start_tdc` (`std_logic`)	`use_data_rate` asserted during FD data phase; `start_tdc` pulsed at BRS to begin delay measurement

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.3	`PCS_Data.Indicate`	`Input_Unit`	Indicate bus polarity at sample point	`PCS -> MAC`	`mac_pcs_if.bus_polarity` (`std_logic`), `mac_pcs_if.sp` (`std_logic`), `mac_pcs_if.ssp` (`std_logic`), `mac_pcs_if.fifo_index` (`t_fifo_index_vec`)	sp pulses once per nominal sample point; ssp pulses once per secondary sample point for TDC
7.2.1, 7.2.6	`PCS_Status.Receiver`	`D_Receive`	Indicate FD data-phase interval to PCS	`MAC -> PCS`	not yet implemented	Asserted during FD data-phase reception

Table 10: Interface definition for `aui_if`. All signals are `std_logic`, as this interface crosses from the ISO protocol domain into the physical medium.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.4.2.1	`output symbol`	`Dominant/recessive symbol`	Drive physical output symbol	`PCS -> PMA`	`aui_if.tx` (`std_logic`)	Updated on each `Output_Unit` request
7.4.2.2, 8.1.3.4	`bus_off symbol`	`Bus-off control`	Switch node off bus	`PCS -> PMA`	`aui_if.bus_off` (`std_logic`)	On `Bus_off_request` from FCE
7.4.2.3, 8.1.3.4	`bus_off_release symbol`	`Bus-off release control`	Release node from bus-off	`PCS -> PMA`	`aui_if.bus_off_release` (`std_logic`)	On `Bus_off_release_request` from FCE
7.4.3	`input symbol`	`Dominant/recessive symbol`	Indicate physical input symbol	`PMA -> PCS`	`aui_if.rx` (`std_logic`)	Continuously driven by PMA

Table 11: Interface definition for `fce_llc_if`. Carries bus-off status and mode-request handshake between the FCE and LLC layers [1, Sec. 8.1.3.2].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	`Normal_mode_request`	`Mode request`	Request reset to normal mode	`LLC -> FCE`	`fce_llc_if.normal_mode_request` (`std_logic`)	Issued on startup/restart
8.1.3.2	`Normal_mode_response`	`Mode response`	Acknowledge normal-mode request	`FCE -> LLC`	`fce_llc_if.normal_mode_response` (`std_logic`)	Returned after FCE processing
8.1.3.2	`Bus_off`	`Bus-off status`	Indicate node is bus-off	`FCE -> LLC`	`fce_llc_if.bus_off` (`std_logic`)	Asserted on bus-off transition

Table 12: Interface definition for `fce_mac_if`. The MAC reports error events and frame outcomes to the FCE; the FCE returns the current error-active/passive state to the MAC [1, Sec. 8.1.3.3].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	`Transmit/receive`	`Transfer mode context`	Report current TX/RX context	`MAC -> FCE`	`fce_mac_if.transmitting` (`std_logic`)	Updated with MAC transfer context
8.1.3.3	`Error`	`Error event`	Report detected protocol error	`MAC -> FCE`	`fce_mac_if.error` (`std_logic`)	Pulse on bit/stuff/CRC/form/ACK error
8.1.3.3	`Primary_error`	`Primary error event`	Report primary error condition	`MAC -> FCE`	`fce_mac_if.primary_error` (`std_logic`)	Pulse on primary error condition
8.1.3.3	`Error/overload flag`	`EF/OF state`	Report EF/OF transmission state	`MAC -> FCE`	`fce_mac_if.sending_error_overloa` (`std_logic`)	Asserted during EF/OF transmission
8.1.3.3	`Counters_unchanged`	`Counter-update qualifier`	Qualify counter exception path	`MAC -> FCE`	`fce_mac_if.counters_unchanged` (`std_logic`)	Asserted on rule-c exception cases
8.1.3.3	`Error_delimiter_too_la`	`Late delimiter event`	Report late error-delimiter condition	`MAC -> FCE`	`fce_mac_if.error_delimiter_too_l` (`std_logic`)	Asserted on late delimiter condition

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	`Successful_transfer`	`Transfer completion event`	Report successful TX/RX completion	`MAC -> FCE`	`fce_mac_if.successful_transfer` (`std_logic`)	Pulse on successful frame transfer
8.1.3.3	`Error_passive_response`	`State response`	Report entry into error-passive state	`MAC -> FCE`	`fce_mac_if.error_passive_respons` (`std_logic`)	On state transition completion
8.1.3.3	`Error_active_response`	`State response`	Report return to error-active state	`MAC -> FCE`	`fce_mac_if.error_active_response` (`std_logic`)	On state transition completion
8.1.3.3	`Error_passive_request`	`State request`	Request MAC enter error-passive state	`FCE -> MAC`	`fce_mac_if.error_passive_request` (`std_logic`)	On TEC/REC threshold crossing
8.1.3.3	`Error_active_request`	`State request`	Request MAC return to error-active state	`FCE -> MAC`	`fce_mac_if.error_active_request` (`std_logic`)	On TEC/REC recovery

Table 13: Interface definition for `fce_pcs_if`. Carries bus-off and bus-off-release request/response handshakes between the FCE and PCS layers [1, Sec. 8.1.3.4].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.4	`Bus_off_request`	`Bus-off request`	Request node switch-off from bus	`FCE -> PCS`	`fce_pcs_if.bus_off_request` (`std_logic`)	On bus-off transition condition
8.1.3.4	`Bus_off_release_request`	`Bus-off release request`	Request node re-enable from bus-off	`FCE -> PCS`	`fce_pcs_if.bus_off_release_requ` (`std_logic`)	On restart/reintegration

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.4	`Bus_off_response`	`Bus-off response`	Acknowledge bus-off request	`PCS -> FCE`	`fce_pcs_if.bus_off_response` (`std_logic`)	Returned after bus-off action
8.1.3.4	`Bus_off_release_respons`	`Bus-off release response`	Acknowledge bus-off-release request	`PCS -> FCE`	`fce_pcs_if.bus_off_release_resp` (`std_logic`)	Returned after release action

4.5 Protocol-Driven Type System

Two VHDL packages centralize the shared definitions used across the design. All module interfaces use `std_logic` and `std_logic_vector` exclusively - protocol concepts such as polarity, frame format, and transfer status are encoded as named constants. Enumeration types such as FSM state types are declared locally within each module's architecture, keeping the packages free of module-specific types.

`pk_can_types` (`can_types_p.vhd`) is the primary design package, organized into the following sections:

1. **Protocol constants** - bus polarity (`c_dominant` = '0', `c_recessive` = '1'), frame field widths (base ID 11 bits, extended ID 18 bits, DLC 4 bits, EOF 7 bits), bit stuffing parameters (`c_stuff_width` = 5), CRC polynomial vectors and selector constants (CRC-15, CRC-17, CRC-21), transfer status encodings (`c_ongoing`, `c_transmitted`, `c_aborted`, `c_lost_arb`, `c_disturbed`), error signalling widths, inter-frame spacing widths, and FCE counter thresholds.
2. **Bit timing subtypes** - constrained natural ranges matching ISO Table 12 [1, Sec. 7.3.2]: prescaler (1-32), propagation segments, phase segments, and SSP offset (1-63).
3. **Composite types** - the sole composite type is `t_llc_metadata`, a record capturing the six LLC config byte fields (IDE, FDF, DLC, FTYP, BRS, ESI) extracted by the serializer and held stable for the duration of a frame.
4. **Interface records** - typed bundles for each inter-module boundary (serializer-FSM, LLC-MAC TX/RX, MAC-PCS, FSM-bit stuffer, FSM-CRC, MAC-FCE, LLC-FCE, FCE-PCS), each paired with a reset constant. These are defined in full in Section 4.4.
5. **LLC frame format** - array types and config byte bit-position constants for both the internal LLC frame format (variable length, streamed to the serializer) and the legacy 71-byte user-facing format.
6. **Utility functions** - `dlc_to_data_length` (ISO Table 5 DLC-to-byte-count conversion), `f_to_gray` (binary-to-Gray encoding for the SBC field), and `f_calc_parity` (XOR parity for SBC).

`can_tb_p` (`can_tb_p.vhd`) is the testbench utility package, extracted from `pk_can_types` to separate simulation-only code from synthesizable definitions. It provides CRC reference calculation, metadata extraction, and bus stream reference model functions used by the testbenches for expected-value computation.

4.6 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

4.6.1 `can_llc_tx`

`can_llc_tx` buffers an incoming LLC frame from the user, streams it byte-by-byte to the MAC serializer, and manages retransmission on bus disturbance up to the ISO-mandated limit [1, Sec. 6.4.5] and 6.5.

4.6.2 `can_llc_rx`

`can_llc_rx` receives a reconstructed LLC frame from the MAC layer, applies acceptance filtering, and delivers accepted frames to the user over the `llc_rx_if` Avalon-ST stream [1, Sec. 6.4.5].

4.7 MAC Sub-layer

The MAC sub-layer is the core of the protocol logic, responsible for bit serialization, CRC generation, bit stuffing, and frame-level error detection. It coordinates closely with the FCE (Section 4.8) for error counter management and node-state transitions (Error Active/Passive/Bus Off), and with the PCS (Section 4.9) for sample-point-driven bit output.

The earlier CAN bus controller concentrated TX, RX, MAC, and FCE logic in a single monolithic FSM (`can_fsm`), with the sub-functions - serialization (`can_ast_to_serial`), bit stuffing (`can_stuff_bit_gen`), and CRC (`gen_crc`) - implemented in satellite modules driven directly by it. PCS timing logic was implemented in `can_node_clock`, which fed sample-point and transmit pulses into `can_fsm` - a strategy retained in the current design.

The current design restructures these modules around the ISO 11898-1 [1] layer boundaries. On the TX side the mapping is direct: `can_ast_to_serial` becomes `can_mac_ser_tx` (Section 4.7.1.3), `can_stuff_bit_gen` becomes `can_mac_bs` (Section 4.7.1.4), `gen_crc` becomes `can_mac_crc` (Section 4.7.1.5), and `can_node_clock` becomes `can_pcs_tx` (Section 4.9.1). The bit stuffer and CRC engine are reused across both paths - each of `can_mac_tx` and `can_mac_rx` instantiates its own copy of the same `can_mac_bs` and `can_mac_crc` entities, with the RX path reusing the bit stuffer for destuffing and the CRC engine for CRC checking. The RX FSM (`can_mac_fsm_rx`) stores received bits directly in an internal frame array and streams the completed frame to the LLC during the quiet phase, eliminating the need for a separate deserializer module. Fault confinement, previously embedded in `can_fsm`, is separated into its own entity (`can_fce`) and wired through the unified `can_mac` wrapper (Section 4.7.3).

4.7.1 `can_mac_tx`

The `can_mac_tx` (Figure 4) is composed of four main components:

1. **can_mac_ser_tx**: Converts LLC bytes into a serial bit stream, extracts LLC metadata from config bytes
2. **can_mac_fsm_tx**: Coordinates frame transmission with per-field state granularity, controls all sub-modules
3. **can_mac_bs**: Shared bit stuffer implementing both dynamic and fixed bit stuffing modes
4. **can_mac_crc**: CRC engine with three parallel generators (CRC-15, CRC-17, CRC-21) and dual data feeds for CC/FD

4.7.1.1 `can_mac_tx` Internal Interfaces The interfaces between MAC sub-components are implementation-defined and carry no direct ISO service primitive mapping. Table 14, Table 15, and Table 16 define each bidirectional bundle; the Direction column identifies the driving component for each field. The bit stuffer and CRC engine interfaces are shared between the TX and RX paths - each path instantiates its own copy wired through identical record types.

Table 14: Interface definition for `can_mac_ser_fsm_if`, connecting `can_mac_ser_tx` and `can_mac_fsm_tx` (see Figure 4).

field	type	direction	description
`data`	`std_logic`	`ser -> fsm`	current bit polarity; FSM reads this when `valid` is asserted
`valid`	`std_logic`	`ser -> fsm`	asserted while a bit is available for the FSM to consume

field	type	direction	description
llc_metadata	t_llc_metadata	ser -> fsm	LLC config byte fields (IDE, FDF, DLC, FTYP, BRS, ESI) extracted by the serializer; valid for the lifetime of the frame
ready	std_logic	fsm -> ser	pulsed when the FSM has consumed the current bit; advances the serializer to the next bit
transfer_status	std_logic_vector(2:0)	fsm -> ser	frame outcome; any non-c_ongoing value terminates serialization

Table 15: Interface definition for can_mac_fsm_bs_if, connecting can_mac_fsm_tx/can_mac_fsm_rx and can_mac_bs (see Figure 4). The bit stuffer is reset by a dedicated bs_rst signal from the FSM rather than a field in this record.

field	type	direction	description
data	std_logic	fsm -> bs	bit polarity fed into the bit stuffer
valid	std_logic	fsm -> bs	pulsed when a new bit is presented to the stuffer
fixed_bit_stuffing_en	std_logic	fsm -> bs	when high, the bit stuffer operates in fixed bit stuffing mode (FD CRC region)
data	std_logic	bs -> fsm	polarity of the required stuff bit
valid	std_logic	bs -> fsm	asserted when a stuff bit insertion is required
stuff_bit_count	std_logic_vector(3:0)	bs -> fsm	gray-coded stuff bit count with parity for the FD SBC field

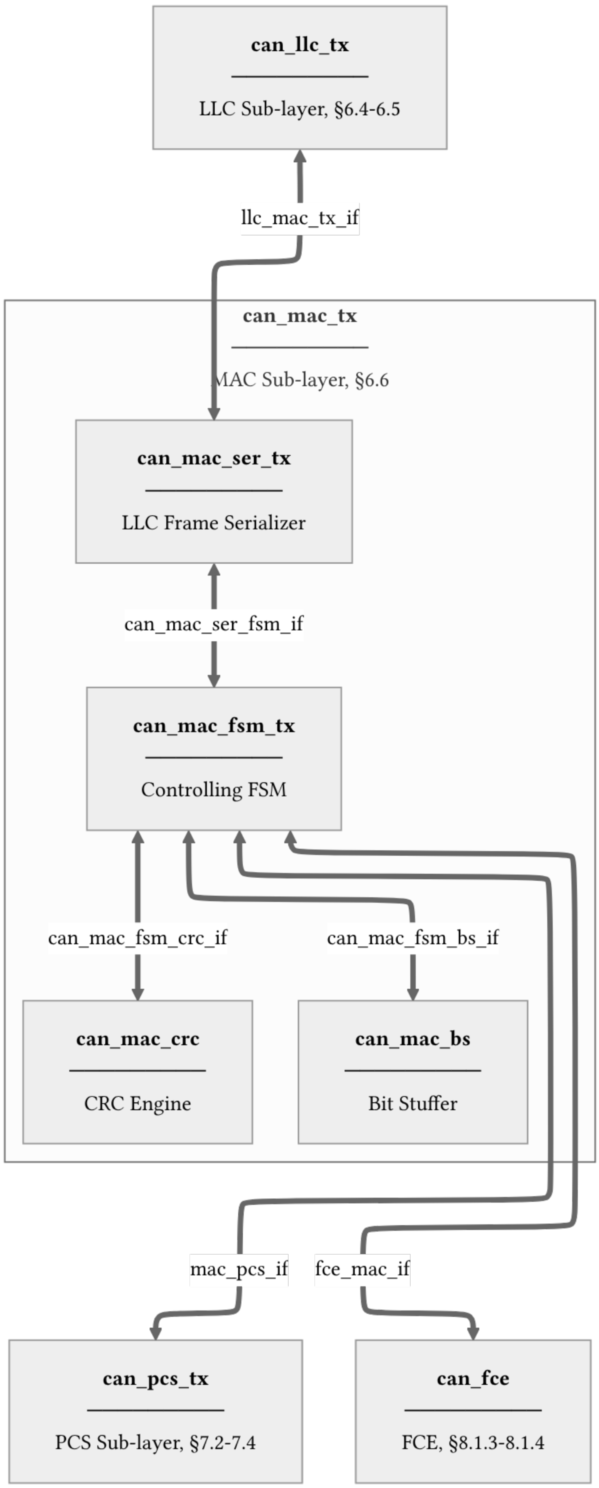

Figure 4: can_mac_tx architecture showing internal component interconnections and external interfaces. Internal interface definitions are provided for can_mac_ser_fsm_if (Table 14), can_mac_fsm_bs_if (Table 15), and can_mac_fsm_crc_if (Table 16). The bit stuffer and CRC engine are the same entities reused in can_mac_rx.

Table 16: Interface definition for `can_mac_fsm_crc_if`, connecting `can_mac_fsm_tx`/`can_mac_fsm_rx` and `can_mac_crc` (see Figure 4). Separate data feeds for CC and FD are required because CC and FD frames compute CRC over different bit streams: CC excludes stuff bits while FD includes them [1, Sec. 10.4.2.6]. This dual-feed design also allows the RX path to run both CRC engines in parallel until the frame type is known. The CRC engine is reset by a dedicated `crc_rst` signal from the FSM.

field	type	direction	description
`crc_poly_select`	`std_logic_vector(1:0)`	`fsm -> crc`	selects the active CRC polynomial: CRC-15, CRC-17, or CRC-21
`valid_cc`	`std_logic`	`fsm -> crc`	pulsed when `data_cc` should be accumulated into the CRC-15 engine
`valid_fd`	`std_logic`	`fsm -> crc`	pulsed when `data_fd` should be accumulated into the CRC-17/CRC-21 engines
`data_cc`	`std_logic`	`fsm -> crc`	bit value fed to the CRC-15 engine (CC frames: data bits only, no stuff bits)
`data_fd`	`std_logic`	`fsm -> crc`	bit value fed to the CRC-17/CRC-21 engines (FD frames: dynamic stuff bits + data bits)
`crc`	`std_logic_vector(20:0)`	`crc -> fsm`	current CRC register value, left-aligned and zero-padded to 21 bits

4.7.1.2 `can_mac_fsm_tx`

`can_mac_fsm_tx` (Figure 5) orchestrates frame transmission, coordinating the serializer (Section 4.7.1.3), bit stuffer (Section 4.7.1.4), and CRC engine (Section 4.7.1.5). The FSM uses per-field state granularity: rather than a single `s_transmitting_frame` state with function-dispatched field logic, each protocol field (SOF, ID, DLC, data, CRC, ACK, EOF, etc.) has its own dedicated FSM state. This design makes the field boundaries explicit in the state encoding and eliminates the need for frame parameter precomputation functions. The FSM derives `data_len` and `crc_length` from `t_llc_metadata` on frame entry and tracks `bit_count` within each field state.

On each sample-point strobe from `can_pcs_tx` (Section 4.9.1), each frame-field state executes the following sequence:

1. **Monitor the bus.** The sampled bus polarity is compared against the previously transmitted bit to detect errors, ACK, and arbitration loss. In the CAN-FD data phase, the FSM maintains a 32-bit polarity history shift register for Transmitter Delay Compensation (TDC). Any pending secondary sample point (SSP) observation from the previous bit period is evaluated against this history before the current bit is checked [1, Sec. 7.3.4]. A detected error triggers a transition to `s_error_overload` with the appropriate flag type based on the FCE fault confinement status [1, Secs. 8.1.3–8.1.4]. Arbitration

loss during the ID or control fields causes the FSM to stop driving and return to `s_intermission`.

2. **Determine the next bit.** If the bit stuffer has a pending stuff bit (`bs_i.valid`), that takes priority. Otherwise, the polarity is determined by the current state: form bits (SOF, IDE, FDF, reserved, delimiters, EOF) have fixed polarities, while ID, DLC, data, SBC, and CRC bits are sourced from the serializer, metadata, bit stuffer count, or CRC register respectively.

3. **Feed the CRC engine and bit stuffer.** The output polarity is forwarded to the CRC engine (for bits within the CRC-protected region) via separate `data_cc` and `data_fd` feeds, and to the bit stuffer (for bits within the stuffing region). The FSM asserts `fixed_bit_stuffing_en` when entering the SBC/CRC field region of FD frames to switch the bit stuffer from dynamic to fixed mode.

4. **Present the bit at the PCS interface.** The resolved polarity is driven onto the `t_can_mac_pcs_if_m2s` interface. The `use_data_rate` signal is asserted during the CAN-FD data phase to switch the PCS to faster bit timing, and `start_tdc` is pulsed at the FDF bit to initiate transmitter delay compensation [1, Sec. 7.3.4].

4.7.1.3 `can_mac_ser_tx`

`can_mac_ser_tx` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM (Figure 6) manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization. The serializer extracts LLC metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the two config bytes and registers it in `t_llc_metadata`, which remains stable for the entire frame. The serializer also handles ID field padding - skipping unused bits in the 32-bit ID field for 11-bit base identifiers - and forwards `transfer_status` from the FSM back to the LLC to terminate serialization on error or completion.

4.7.1.4 `can_mac_bs`

`can_mac_bs` implements both dynamic and fixed bit stuffing for CAN Classic and CAN-FD frames [1, Sec. 10.6]. The same entity is instantiated in both `can_mac_tx` and `can_mac_rx` - on the TX side it inserts stuff bits, while on the RX side the FSM uses its output to detect and remove stuff bits from the received stream.

In **dynamic mode** (`fixed_bit_stuffing_en` = ‘0’), the stuffer monitors the polarity stream and inserts an inverse-polarity stuff bit after every five consecutive identical bits (ISO 6.6.13.2). In **fixed mode** (`fixed_bit_stuffing_en` = ‘1’), used for the FD CRC region, one fixed stuff bit (FSB) is inserted on the rising edge of `fixed_bit_stuffing_en`, then one every four real bits (ISO 6.6.13.3.1). Any dynamic stuff bit pending at the mode boundary is suppressed. The stuffer maintains a Gray-coded stuff bit count with parity for the SBC field via `f_to_gray()` and `f_calc_parity()`. The data path diagram is shown in Figure 7.

4.7.1.5 `can_mac_crc`

`can_mac_crc` provides CRC generation and checking for both CAN Classic and CAN-FD frames. CAN Classic frames use CRC-15, while CAN-FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [1, Sec. 10.4.2.6]. The same entity is instantiated in both `can_mac_tx` (for generation) and `can_mac_rx` (for checking). The FSM selects the appropriate polynomial via the `crc_poly_select` field (Table 16). The data path diagram is shown in Figure 8.

Three parallel `gen_crc` instances run continuously on separate data feeds: `data_cc` drives CRC-15 via `valid_cc`, while `data_fd` drives both CRC-17 and CRC-21 via `valid_fd`. This dual-feed architecture is necessary because CC and FD frames compute CRC over different bit streams (CC excludes stuff bits; FD includes them), and the RX path does not know which CRC engine to use until after the frame type has

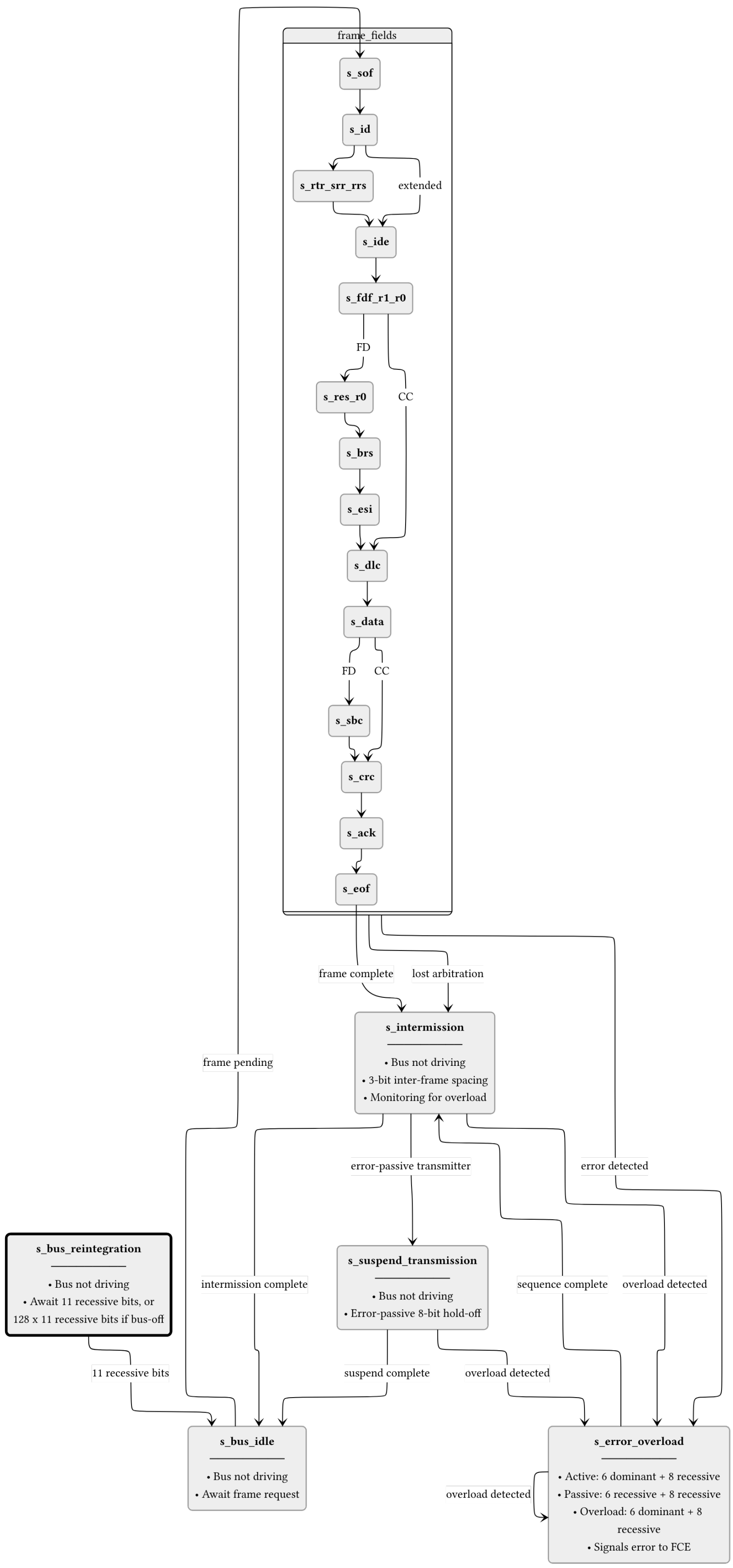

Figure 5: can_mac_fsm_tx FSM with per-field state granularity. Frame-field states (s_sof through s_eof) each handle one protocol field, with bit_count tracking position within the field. After a successful frame or arbitration loss the FSM passes through s_intermission before returning to s_bus_idle. Detected errors branch to s_error_overload, which drives either an active (dominant) or passive (recessive) error flag depending on FCE status. The dashed box groups the frame-field states for clarity.

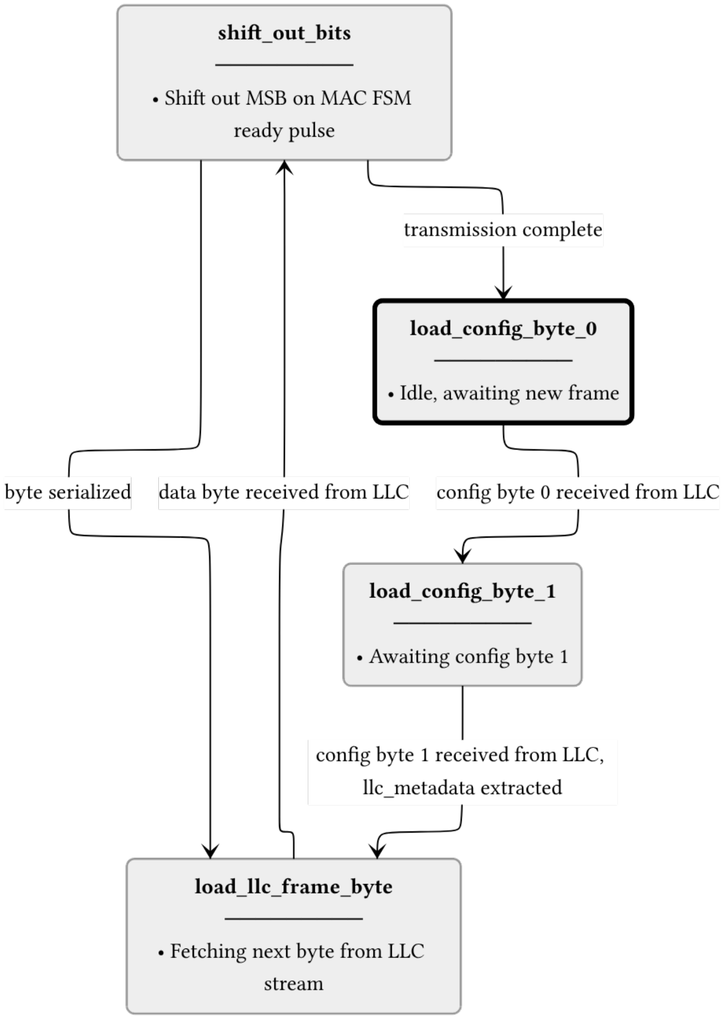

Figure 6: can_mac_ser_tx FSM. The serializer accepts LLC frame bytes over a ready/valid handshake and shifts them out MSB-first to the MAC FSM one bit per ready pulse. LLC metadata is extracted from the two config bytes and registered in t_llc_metadata for use by the FSM throughout the frame.

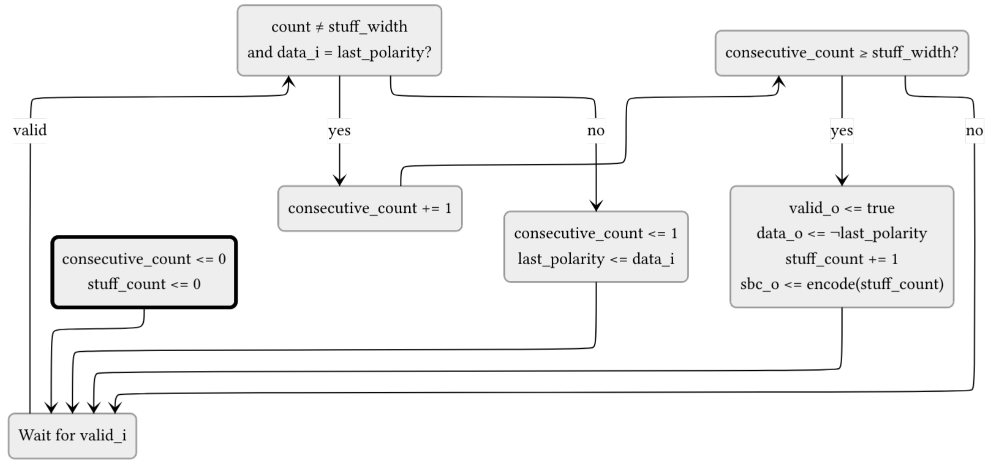

Figure 7: can_mac_bs data path diagram. When valid and data = last_polarity, consecutive_count increments. Otherwise it restarts at 1 with last_polarity updated to data. When consecutive_count reaches stuff_width (5 in dynamic mode, 4 in fixed mode), valid and inverted data are driven and stuff_count increments. The to_gray() + calc_parity() block encodes stuff_count into the stuff_bit_count output. The external rst signal resets consecutive_count and stuff_count to zero. The fixed_bit_stuffing_en signal selects between dynamic and fixed modes. All outputs are registered.

been determined. The output multiplexer selects the active engine's result based on `crc_poly_select` and left-aligns it to the common 21-bit output width.

4.7.2 `can_mac_rx`

`can_mac_rx` receives the bit stream from the PCS, performs bit destuffing and CRC verification, and reconstructs the LLC frame for delivery to `can_llc_rx` [1, Sec. 6.6]. Unlike the TX path, the RX path has no separate deserializer module. The RX FSM (`can_mac_fsm_rx`) stores received bits directly into an internal `llc_frame` array during reception and streams the completed frame byte-by-byte to the LLC during the quiet phase (after EOF and intermission) using a dedicated Avalon-ST streaming process.

The `can_mac_rx` wrapper instantiates three components:

1. **can_mac_fsm_rx**: Frame reception FSM with 17 per-field states mirroring the TX FSM structure
2. **can_mac_bs**: Shared bit stuffer entity, reused for destuffing
3. **can_mac_crc**: Shared CRC engine entity, reused for CRC checking

The RX FSM (Figure 9) validates SBC and CRC fields during reception, detects form errors (reserved bits, CRC mismatch, delimiter errors), and drives ACK dominant during the ACK slot (1 bit for CC, 2 bits for FD). It reports errors to the FCE through the same `t_can_mac_fce_if_m2s` interface used by the TX FSM.

4.7.3 `can_mac` Unified Wrapper

`can_mac` is a structural wrapper that instantiates `can_mac_tx`, `can_mac_rx`, and `can_fce` (Section 4.8), wiring the FCE internally so that the wrapper exposes only LLC and PCS interfaces for each path plus the FCE's LLC and PCS interfaces. TX and RX have separate PCS interfaces (half-duplex; only one path drives the bus at a time). The TX and RX FCE output records (`t_can_mac_fce_if_m2s`) are merged by OR-ing all fields before feeding the FCE input - the half-duplex guarantee ensures no simultaneous assertions from both paths. The FCE's response (`t_can_mac_fce_if_s2m`, carrying `error_passive_request` and `error_active_request`) is fanned back to both TX and RX paths.

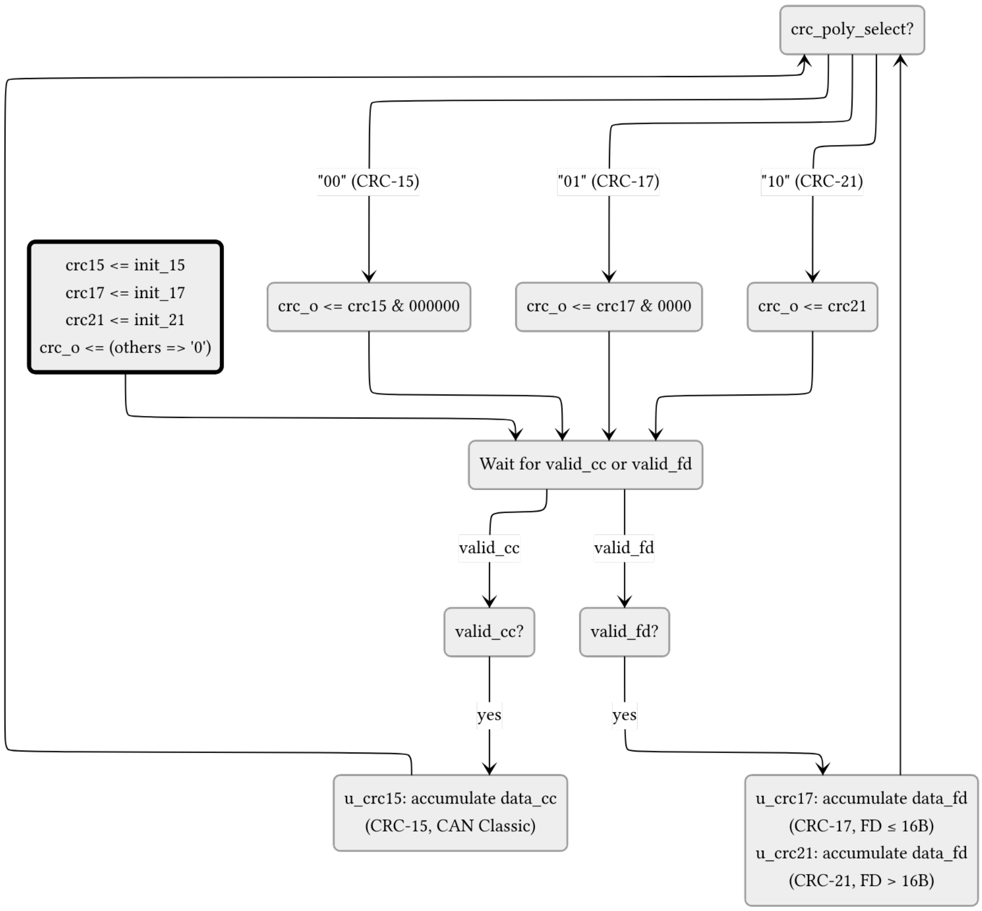

Figure 8: can_mac_crc data path diagram. Three parallel gen_crc instances accumulate continuously: data_cc drives CRC-15 via valid_cc, while data_fd drives both CRC-17 and CRC-21 via valid_fd. The output register selects the active engine result based on crc_poly_select and left-aligns it to 21 bits. The external rst signal reinitializes all three engines and the output register.

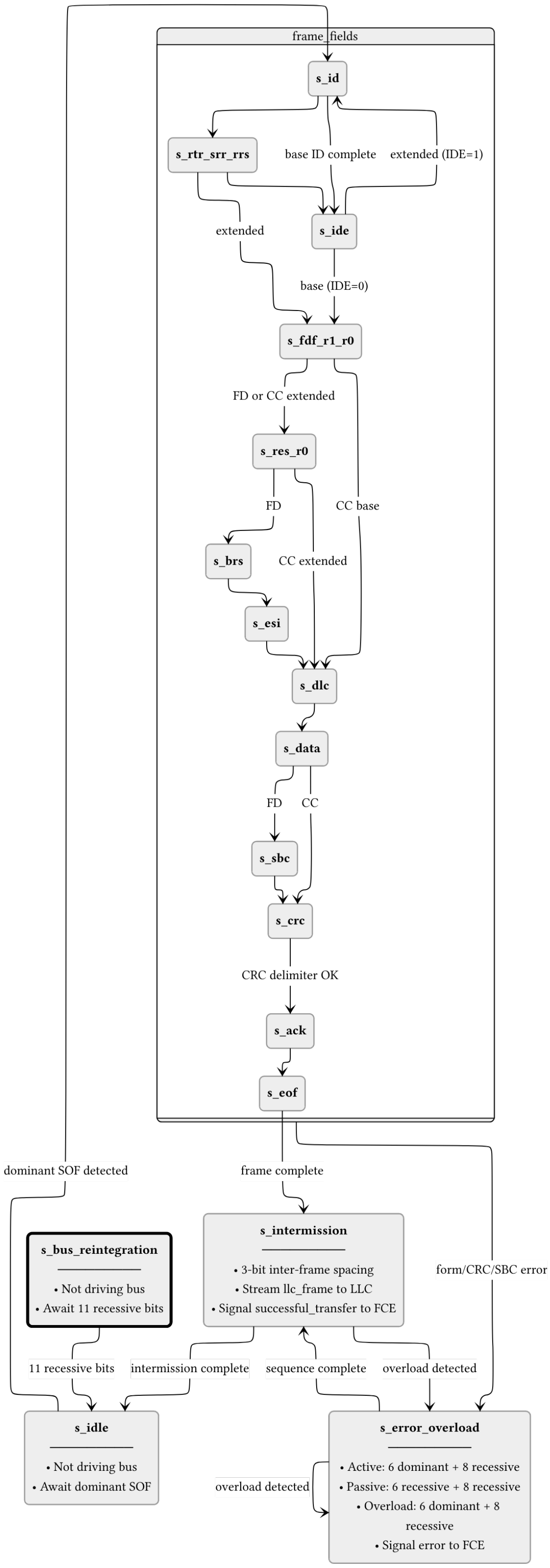

Figure 9: can_mac_fsm_rx FSM with per-field state granularity. The RX FSM mirrors the TX FSM structure (Figure 5) but replaces transmission logic with reception, storage, and validation. On detecting a dominant SOF in s_idle, the FSM enters s_id and stores received bits into an internal llc_frame array. After EOF, the FSM transitions through s_intermission and a dedicated streaming process transfers the stored frame to the LLC. Form errors (reserved bit violations, CRC mismatch, delimiter errors) and SBC mismatches trigger s_error_overload. The dashed box groups the frame-field states for clarity.

4.8 FCE Sub-layer

The Fault Confinement Entity (`can_fce`) implements the error state machine and counter management specified in [1, Secs. 8.1.3–8.1.4]. It maintains two error counters - TEC (Transmitter Error Counter, 0-511) and REC (Receiver Error Counter, 0-255) - and transitions between three states: `s_error_active` (normal operation), `s_error_passive` (TEC or REC > 127), and `s_bus_off` (TEC > 255).

Counter updates follow the ISO 8.1.4.2 rules: TEC increments by 8 on TX errors (unless `counters_unchanged` is asserted), decrements by 1 on successful TX. REC increments by 1 on RX errors during non-error-flag phases, by 8 on primary errors or error-flag-phase errors, and decrements by 1 or clamps to 127 on successful RX. The FCE state machine (Figure 10) transitions between error-active, error-passive, and bus-off based on counter thresholds. Bus-off recovery requires counting 128 idle conditions (11 consecutive recessive bits each) from the PCS via the `idle_condition` signal, or a `normal_mode_request` from the LLC.

4.9 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

4.9.1 `can_pcs_tx`

`can_pcs_tx` handles bit timing and Transmitter Delay Compensation (TDC) for the TX path [1, Secs. 7.2–7.3]. It generates the sample point (SP) strobe used by the MAC FSM to advance frame state, and - when TDC is configured - a secondary sample point (SSP) strobe for data-phase bit monitoring. The FSM (Figure 11) has four states reflecting the two bit rates and the TDC measurement window. The `measuring_delay` state determines the Transmitter Delay Compensation Value (TDCV, [1, Sec. 7.3.4]) by observing the TX-to-RX propagation delay, which together with the configured TDCO offset defines the SSP position for subsequent data-phase bits.

4.9.2 `can_pcs_rx`

`can_pcs_rx` implements bit timing and synchronization for the RX path. It performs hard and soft synchronization on the incoming bus signal and generates the sample point strobe used by the MAC RX layer to latch each received bit [1, Secs. 7.2–7.3].

---

5 Implementation

5.1 Type Safety and Packages

The implementation uses custom record types (defined in `can_types_pkg.vhd`) to ensure clean interfaces between modules.

5.2 Bit Timing and TDC

Detailed description of how `tx_pcs` measures propagation delay and calculates the SSP.

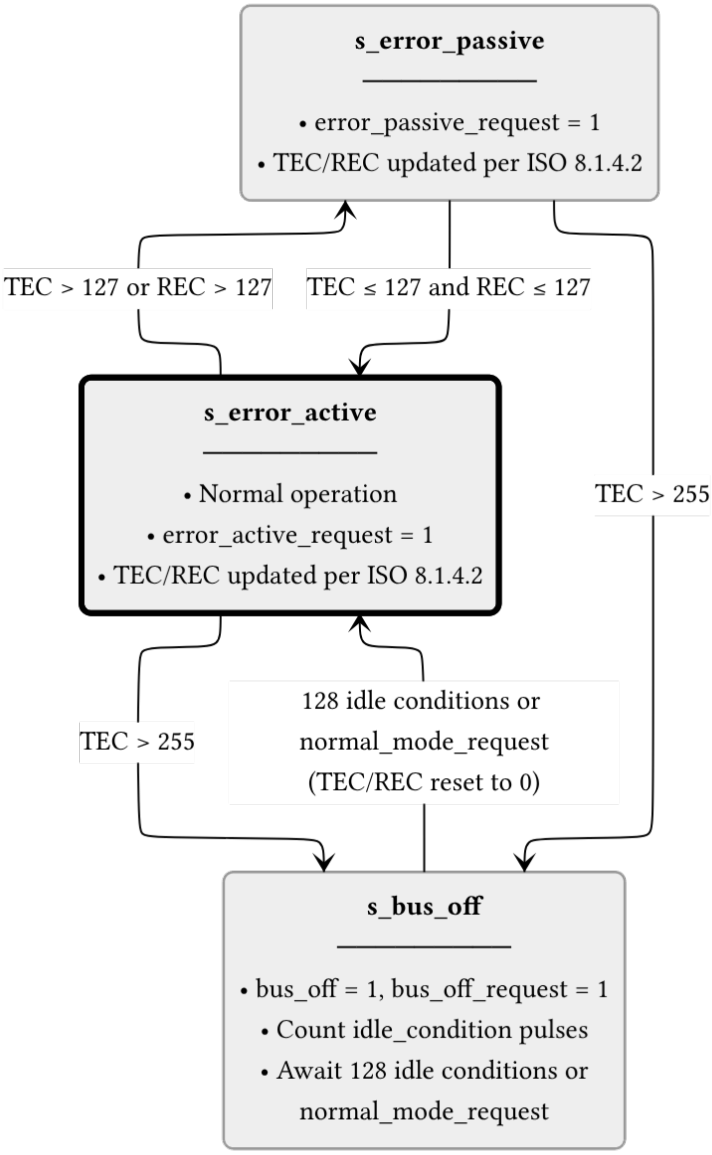

Figure 10: can_fce FSM (ISO 8.1.4.1, Figure 43). The s_error_active state is the normal operating mode. When either TEC or REC exceeds 127, the FCE transitions to s_error_passive and asserts error_passive_request to both MAC paths. If TEC exceeds 255, the node enters s_bus_off: the FCE issues bus_off_request to the PCS and bus_off to the LLC. Recovery from bus-off requires 128 idle conditions (each 11 consecutive recessive bits) counted from the PCS, or a normal_mode_request from the LLC, after which the counters are reset and the FCE returns to s_error_active.

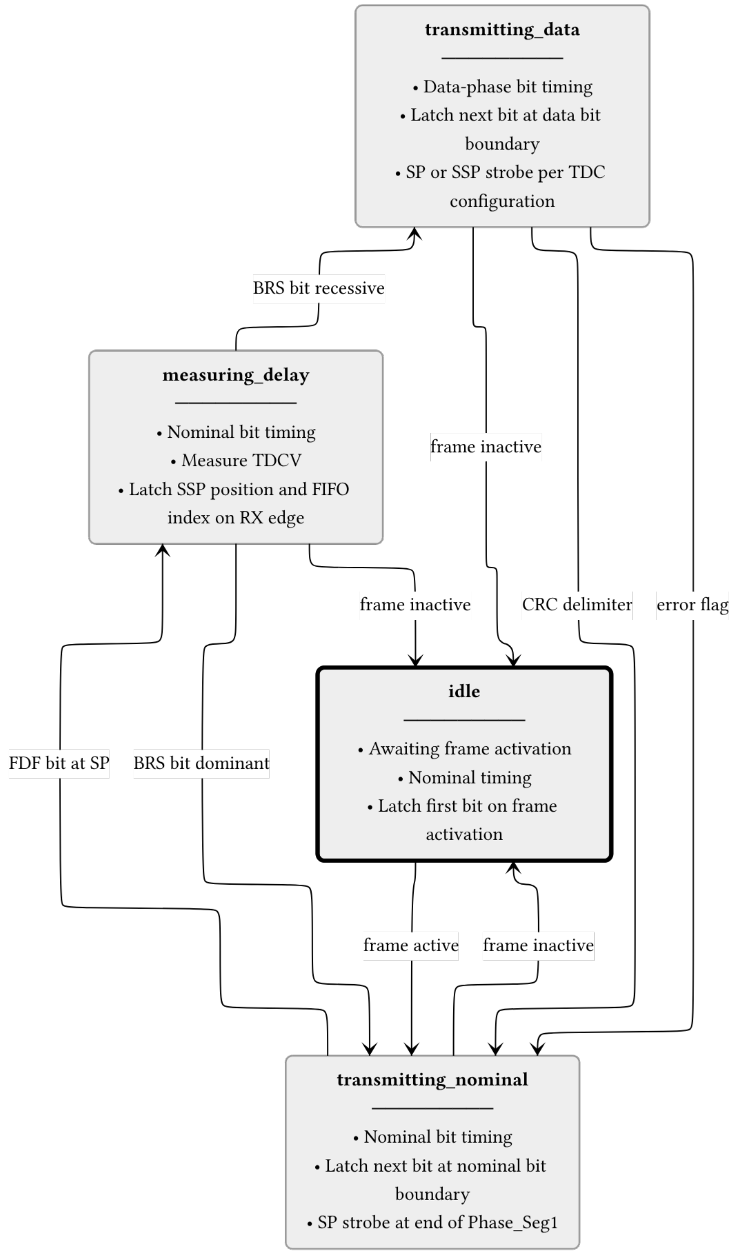

Figure 11: can_pcs_tx FSM. The measuring_delay state is entered on the FDF sample point to measure TDCV (Transmitter Delay Compensation Value, [[1], sec. 7.3.4]). The BRS bit boundary determines whether data-phase timing is used. All non-idle states return to idle when the frame becomes inactive.

5.3 CRC and Bit Stuffing

Implementation details of the flexible CRC generator and the hybrid bit stuffer.

6 Verification and Results

Note: This section is currently being populated as the verification plan is executed.

6.1 Testbench Results Summary

Table 17: Testbench execution status and functional coverage summary.

Testbench	Status	Coverage
`tx_pcs_tb`	Pass	95%
`tx_can_tb`	Pass	88%
`tx_can_protocol_tb`	Pass	92%

---

7 Discussion

Comparison of the implemented architecture against theoretical models. Performance analysis in high-load scenarios.

8 Conclusion

Summary of work completed and how objectives were met.

9 References

[1] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer.* 2024.

[2] M. Jeřábek and P. Píša, “CTU CAN FD — open-source CAN FD IP core.” 2019. Available: https://gitlab.fel.cvut.cz/canbus/ctucanfd_ip_core

[3] M. Jeřábek, “Open-source and open-hardware CAN FD protocol support,” PhD thesis, Czech Technical University in Prague, 2019. Available: https://dspace.cvut.cz/handle/10467/82630

[4] International Organization for Standardization, *Road vehicles — controller area network (CAN) conformance test plan — Part 1: Data link layer and physical signalling.* 2016.

[5] OpenCores Contributors, “CAN protocol controller — OpenCores.” 2003. Available: https://opencores.org/projects/can

[6] S. V. Nøsen, “Canola — CAN controller in VHDL.” 2020. Available: https://github.com/svnesbo/canola

[7] Robert Bosch GmbH, “M_CAN — CAN FD protocol controller IP module.” 2024. Available: https://www.bosch-semiconductors.com/ip-modules/can-fd/

[8] F. Hartwich, “CAN with flexible data-rate,” in *Proceedings of the 13th international CAN conference (iCC 2012)*, 2012.

[9] AMD (Xilinx), "CAN FD controller — Vivado design suite product guide (PG223)." 2024. Available: https://docs.amd.com/r/en-US/pg223-canfd

[10] CAST, Inc., "CAN-FD protocol controller." 2024. Available: https://www.cast-inc.com/automotive/can/can-fd-protocol-controller

[11] C. A. R. Hoare, "An axiomatic basis for computer programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, Oct. 1969, doi: 10.1145/363235.363259.

[12] B. Alpern and F. B. Schneider, "Defining liveness," *Information Processing Letters*, vol. 21, no. 4, pp. 181–185, Oct. 1985, doi: 10.1016/0020-0190(85)90056-0.